

SIMATS ENGINEERING
CO4 ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking

COURSE CODE: ITA1410

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01

Total Marks:100

Annexure A

SECURITY TESTING SIMULATION TASK

Code of Conduct:

I S Venkata Abhishek(192311053) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Abhishek

QUESTION 1: WEB SERVER RECONNAISSANCE AND VULNERABILITY ANALYSIS

1. Objective

The objective of this experiment is to perform reconnaissance and vulnerability analysis of an authorized web server. The assessment includes information gathering, network reconnaissance, port and service identification, web technology detection, HTTP header analysis, web-server vulnerability scanning, and directory enumeration.

The following tools were used:

- Google/Search Engine
- WHOIS
- Ping
- Traceroute/Tracert
- IPConfig/Ifconfig
- Netstat/ss
- Nmap
- WhatWeb
- Curl
- Nikto
- Gobuster/Dirb

2. Target Information

Target IP: <TARGET_IP>

Target URL: <TARGET_URL>

Example:

Target IP : 192.168.56.101

Target URL : http://192.168.56.101

The above example is only a laboratory example. The actual faculty-provided target should be used during implementation.

3. Google Reconnaissance

Google/search-engine reconnaissance was used to understand publicly available information related to the authorized laboratory domain.

Example search:

site:<AUTHORIZED_DOMAIN>

Other useful searches within the authorized scope:

site:<AUTHORIZED_DOMAIN> login

site:<AUTHORIZED_DOMAIN> admin

site:<AUTHORIZED_DOMAIN> filetype:pdf

Observation

Search-engine reconnaissance can reveal information such as indexed pages, login pages, documentation, and publicly visible resources. Only information belonging to the authorized assessment scope should be recorded.

4. WHOIS Information Gathering

Command:

whois <AUTHORIZED_DOMAIN>

Observation

WHOIS may provide domain registration information, registrar details, name servers, and registration dates depending on the domain and privacy settings.

Security Impact

Excessive publicly available registration or infrastructure information can help an attacker understand the organization and its external infrastructure.

Recommendation

Organizations should minimize unnecessary information exposure and use appropriate domain-registration privacy and organizational security controls.

5. Basic Network Reconnaissance

5.1 Ping

Command:

```
ping -c 4 <TARGET_IP>
```

Observation

The ping command was used to determine whether the authorized target was reachable and to observe response time.

Example:

```
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.5 ms
```

```
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.4 ms
```

Result

The target responded to ICMP requests and was reachable from the laboratory machine.

5.2 Traceroute

On Kali Linux:

```
traceroute <TARGET_IP>
```

If traceroute is unavailable:

```
tracert <TARGET_IP>
```

Observation

Traceroute identifies the network path between the testing system and the authorized target.

5.3 IP Configuration

Command:

```
ip addr
```

or:

```
ifconfig
```

Observation

The command displays the local IP address, network interface, subnet information, and MAC address.

5.4 Netstat

Command:

```
netstat -tulnp
```

If netstat is unavailable:

```
ss -tulnp
```

Observation

The command displays listening TCP/UDP services on the testing system.

6. Nmap Port Scanning

Basic scan:

```
nmap <TARGET_IP>
```

Service-version detection:

```
nmap -sV <TARGET_IP>
```

Operating-system detection:

```
sudo nmap -O <TARGET_IP>
```

Default script scanning:

```
nmap -sC <TARGET_IP>
```

Combined authorized lab scan:

```
sudo nmap -sC -sV -O <TARGET_IP>
```

Example Observation

A sample result may look like:

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

22/tcp	open	ssh	OpenSSH
--------	------	-----	---------

80/tcp	open	http	Apache httpd
--------	------	------	--------------

443/tcp open https Apache httpd

The actual result must be taken from the assigned laboratory target.

Analysis

Port	Service	Possible Purpose	Security Concern
22	SSH	Remote administration	Brute-force attempts and outdated SSH versions
80	HTTP	Web service	Unencrypted HTTP communication
443	HTTPS	Secure web service	TLS configuration must be secure

Only services actually discovered during the scan should be included in the final report.

7. WhatWeb Technology Detection

Command:

```
whatweb <TARGET_URL>
```

Example:

```
whatweb http://192.168.56.101
```

Observation

WhatWeb can identify technologies such as:

- Web server
- Programming language
- CMS
- JavaScript libraries
- Frameworks
- Web application technologies

Security Impact

Technology fingerprinting may help attackers identify outdated software and search for known vulnerabilities.

Recommendation

Avoid unnecessary disclosure of software versions and maintain all server-side components with current security patches.

8. HTTP Header Analysis Using Curl

Command:

```
curl -I <TARGET_URL>
```

For example:

```
curl -I http://192.168.56.101
```

To inspect the complete response:

```
curl -i <TARGET_URL>
```

Headers to Examine

Important headers include:

Server

X-Powered-By

Content-Security-Policy

Strict-Transport-Security

X-Frame-Options

X-Content-Type-Options

Referrer-Policy

Set-Cookie

Example Observation

If the response contains:

Server: Apache/2.4.x

X-Powered-By: PHP/8.x

the server may disclose unnecessary software information.

Recommendation

Remove or minimize unnecessary version information and configure appropriate security headers.

9. Nikto Web-Server Scan

Command:

```
nikto -h <TARGET_URL>
```

Example:

```
nikto -h http://192.168.56.101
```

Nikto Checks

Nikto can identify:

- Dangerous files
- Default files
- Misconfigured web servers
- Missing security headers
- Outdated server components
- CGI-related issues
- Information disclosure

Observation

The output should be saved as evidence:

```
nikto -h <TARGET_URL> -output nikto_report.txt
```

10. Gobuster Directory Enumeration

Command:

```
gobuster dir -u <TARGET_URL> -w /usr/share/wordlists/dirb/common.txt
```

If the wordlist is available at another location, use the appropriate laboratory wordlist.

Example

```
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt
```

Observation

Gobuster may identify resources such as:

/admin

/login

/uploads

/backup

/config

Only resources discovered on the authorized laboratory server should be examined.

11. Dirb Alternative

Command:

```
dirb <TARGET_URL>
```

Example:

```
dirb http://192.168.56.101
```

12. Finding Classification

Finding	Severity	Impact
Unnecessary server-version disclosure	Low	Helps fingerprint technology
Missing security headers	Low/Medium	Reduces browser-side protection
Exposed backup file	High	May disclose source/configuration information
Exposed configuration file	High/Critical	May contain sensitive credentials
Outdated web component	Medium/High	May contain known vulnerabilities

Unprotected administrative page	Medium/High	May expose privileged functionality
--	-------------	-------------------------------------

The final severity should be based on the actual evidence obtained from the assigned target.

13. Recommendations

1. Remove unnecessary server and framework version information.
2. Disable directory listing where it is not required.
3. Remove backup, temporary, and unused files.
4. Protect administrative interfaces using authentication and authorization.
5. Keep web-server software and frameworks updated.
6. Configure appropriate HTTP security headers.
7. Disable unnecessary services and ports.
8. Use HTTPS for sensitive communication.
9. Protect configuration files from direct access.
10. Perform periodic vulnerability assessments.

QUESTION 2: WEB APPLICATION VULNERABILITY ASSESSMENT

1. Objective

The objective is to assess an authorized vulnerable web application for common security weaknesses including SQL Injection, Cross-Site Scripting, authentication problems, session-management weaknesses, directory traversal, and security misconfiguration.

The testing was restricted to the isolated laboratory application.

2. Application Reconnaissance

First, identify the application technologies.

whatweb <TARGET_URL>

Nmap service detection:

```
nmap -sC -sV <TARGET_IP>
```

HTTP response:

```
curl -I <TARGET_URL>
```

3. Browser Developer Tools

The browser Developer Tools were used to inspect:

- HTTP requests
- HTTP responses
- Parameters
- Cookies
- Forms
- JavaScript files
- API endpoints
- Session information

The Network tab was used to identify requests generated while interacting with the authorized application.

4. Burp Suite

Start Burp Suite:

```
burpsuite
```

Basic Configuration

1. Open Burp Suite.
2. Select the temporary project option.
3. Open the Proxy section.
4. Configure the browser to use the Burp proxy.
5. Browse only the authorized laboratory application.
6. Capture the generated HTTP requests.
7. Send selected requests to Repeater for safe testing.

5. SQL Injection Testing

A laboratory input field or parameter was selected for testing.

Example harmless test input:

'

Other basic syntax-testing input:

"

The objective is to determine whether unexpected database-related errors or abnormal responses occur.

Observation

If an application returns database errors after specially crafted input, the parameter may require further investigation.

Impact

SQL Injection can potentially allow unauthorized database access, modification, or disclosure.

Recommendation

Use:

- Parameterized queries
- Prepared statements
- Input validation
- Least-privilege database accounts
- Safe error handling

6. Cross-Site Scripting Testing

A harmless XSS test payload can be used only in the authorized laboratory:

<script>alert(1)</script>

The payload should be entered into an approved test field.

Observation

If the application executes the script instead of safely encoding it, the input may be vulnerable to XSS.

Impact

XSS can allow malicious JavaScript to execute in another user's browser and may affect confidentiality, integrity, and session security.

Recommendation

Use:

- Context-aware output encoding
- Input validation
- Content Security Policy
- Secure cookie settings
- Framework-provided escaping mechanisms

7. Authentication Assessment

The login mechanism was examined without performing unauthorized password attacks.

The following were checked:

- Password requirements
- Error messages
- Logout functionality
- Session expiration
- Access after logout
- Authentication enforcement

Observation

If protected resources remain accessible after logout or without authentication, this indicates a possible authentication/session weakness.

Recommendation

Implement strong authentication, secure session expiration, server-side authentication checks, and appropriate account protection mechanisms.

8. Session Management Assessment

Cookies were inspected using browser Developer Tools or Burp Suite.

Important attributes:

Secure

HttpOnly

SameSite

Example

A secure session cookie should ideally include:

Secure

HttpOnly

SameSite

Impact

Poor cookie configuration can increase the risk of session theft or cross-site attacks.

9. Directory Traversal Assessment

A permitted laboratory parameter was tested with a controlled traversal sequence such as:

../

The test was limited to determining whether the application incorrectly processes path traversal.

Observation

If unauthorized files become accessible, the application may contain a directory traversal vulnerability.

Recommendation

Use:

- Canonical path validation
- Allow-list validation

- Safe file APIs
- Restriction of accessible directories
- Proper server-side authorization

10. Nikto

Command:

```
nikto -h <TARGET_URL>
```

Nikto output was saved:

```
nikto -h <TARGET_URL> -output q2_nikto.txt
```

11. OWASP ZAP

OWASP ZAP was used to crawl and assess the authorized application.

The application URL was entered into the ZAP interface and the passive scan results were reviewed.

Testing was restricted to the laboratory environment.

Observation

Potential alerts were reviewed and classified according to actual evidence rather than assuming every automated alert was a confirmed vulnerability.

12. Vulnerability Classification

Vulnerability	Severity	Possible Impact
SQL Injection	High/Critical	Database disclosure/manipulation
Stored XSS	High	Persistent script execution
Reflected XSS	Medium	Script execution through crafted input
Weak session management	Medium/High	Account/session compromise
Directory Traversal	High	Unauthorized file access

Missing security configuration	Low/Medium	Reduced security protection
---------------------------------------	------------	-----------------------------

QUESTION 3: SECURITY HEADER AND HTTPS CONFIGURATION ASSESSMENT

1. Objective

The objective is to assess HTTP security headers and HTTPS/TLS configuration of the authorized laboratory web application.

2. HTTP Header Analysis

Command:

```
curl -I http://<TARGET_IP>
```

For HTTPS:

```
curl -I https://<TARGET_IP>
```

Complete headers:

```
curl -k -I https://<TARGET_IP>
```

The -k option should only be used during controlled laboratory testing when the lab uses a self-signed certificate.

3. Security Headers Checked

The following headers were examined:

Content-Security-Policy

Content-Security-Policy

Purpose: Helps control which sources of content the browser is permitted to load.

Strict-Transport-Security

Strict-Transport-Security

Purpose: Instructs browsers to use HTTPS for future connections.

X-Frame-Options

X-Frame-Options

Purpose: Helps prevent clickjacking.

X-Content-Type-Options

X-Content-Type-Options

Purpose: Helps prevent MIME-type sniffing.

Referrer-Policy

Referrer-Policy

Purpose: Controls information sent through the HTTP Referer header.

4. HTTP to HTTPS Redirection

Command:

```
curl -I http://<TARGET_IP>
```

Expected Secure Behaviour

The response should redirect HTTP to HTTPS, for example:

HTTP/1.1 301 Moved Permanently

Location: https://<TARGET_IP>/

The exact response depends on the server configuration.

5. Nmap SSL/TLS Assessment

Basic SSL script:

```
nmap --script ssl-enum-ciphers -p 443 <TARGET_IP>
```

Certificate information:

```
nmap --script ssl-cert -p 443 <TARGET_IP>
```

Combined:

```
nmap --script ssl-enum-ciphers,ssl-cert -p 443 <TARGET_IP>
```

Information Examined

- TLS protocol versions

- Supported cipher suites
- Certificate subject
- Certificate validity
- Certificate issuer
- Certificate expiration
- Weak cryptographic configurations

6. TLS Finding Classification

Finding	Severity	Impact
TLS 1.0/1.1 enabled	Medium	Uses outdated protocols
Weak cipher suites	Medium/High	Reduced confidentiality
Expired certificate	High	Trust and security problems
Self-signed certificate in production	Medium/High	Certificate trust issue
HTTP not redirected to HTTPS	Medium	Users may communicate without encryption
Missing HSTS	Medium	Reduced HTTPS enforcement
Missing CSP	Medium	Reduced XSS protection
Missing X-Frame-Options	Low/Medium	Increased clickjacking risk
Missing X-Content-Type-Options	Low	MIME-sniffing risk

7. Recommendations

1. Disable obsolete TLS protocols.
2. Use modern TLS configurations.
3. Disable weak cipher suites.
4. Use a valid certificate from a trusted certificate authority for production.
5. Configure automatic HTTP-to-HTTPS redirection.

6. Enable HSTS after confirming HTTPS is correctly configured.
7. Implement an appropriate Content Security Policy.
8. Configure X-Frame-Options or an appropriate CSP frame-ancestors policy.
9. Enable X-Content-Type-Options.
10. Monitor certificate expiration and renew certificates before expiry.

QUESTION 4: BROKEN ACCESS-CONTROL ASSESSMENT

1. Objective

The objective is to determine whether a normal student account can access another student's records or administrator-only resources.

Testing was performed only with the student and administrator test accounts and object identifiers provided by the faculty.

2. Authentication and Authorization

Authentication answers:

"Who is the user?"

Authorization answers:

"What is the authenticated user allowed to access?"

A user may be correctly authenticated but still be unauthorized to access another student's records or administrator resources.

3. OWASP ZAP Procedure

1. Start OWASP ZAP.
2. Configure the browser to use the ZAP proxy.
3. Log in using the faculty-provided student test account.
4. Access the student's permitted record.
5. Capture the HTTP request.
6. Identify object identifiers in the request.

7. Use only the faculty-provided test identifiers.
8. Send the request to the appropriate ZAP testing area.
9. Compare the response for authorized and unauthorized objects.
10. Test administrator-only resources using the student account.
11. Record the response status and application behaviour.

4. Example Request Analysis

A request may contain an object identifier similar to:

GET /student?id=101 HTTP/1.1

If the faculty provides another test identifier, the request can be compared using that authorized identifier.

Example:

GET /student?id=102 HTTP/1.1

The test should determine whether the server correctly checks ownership.

5. Evidence of Secure Behaviour

A properly protected application may return:

HTTP/1.1 403 Forbidden

or:

HTTP/1.1 404 Not Found

or another appropriate denial response.

The application must not disclose another student's protected information.

6. Possible Vulnerability

If a normal student account changes an authorized object identifier and receives another student's confidential record, the application may contain an Insecure Direct Object Reference (IDOR) or Broken Object Level Authorization vulnerability.

Example Finding

Student A → Student A record = Allowed

Student A → Student B record = Denied

Student A → Admin resource = Denied

This is the expected secure behaviour.

If instead:

Student A → Student B record = Allowed

then the access-control mechanism requires remediation.

7. Severity

Finding	Severity	Impact
Student accesses another student's record	High	Confidential student information disclosure
Student accesses administrator page	High/Critical	Unauthorized privileged functionality
Missing server-side authorization	High	Unauthorized resource access
Predictable object identifiers alone	Low/Medium	May facilitate enumeration

Severity must be based on the actual data and functionality exposed.

8. Recommendations

1. Perform authorization checks on every protected request.
2. Never rely only on hidden fields or client-side controls.
3. Validate the authenticated user's ownership of requested objects.
4. Enforce role-based authorization on the server.
5. Use least-privilege access.
6. Avoid exposing unnecessary sequential identifiers.

7. Use indirect/randomized object references where appropriate.
8. Log unauthorized access attempts.
9. Test authorization for every API endpoint.
10. Apply deny-by-default access-control policies.

QUESTION 5: AUTOMATED SCANNING AND MANUAL VALIDATION

1. Objective

The objective is to compare automated vulnerability-scanning results from Nikto and OWASP ZAP and manually validate selected findings within the authorized laboratory environment.

2. Nikto Scan

Command:

```
nikto -h <TARGET_URL>
```

Save the result:

```
nikto -h <TARGET_URL> -output nikto_q5.txt
```

Information Obtained

Nikto can report:

- Web-server misconfigurations
- Missing headers
- Default files
- Potentially dangerous files
- Outdated components
- Information disclosure

3. OWASP ZAP Assessment

OWASP ZAP was used to crawl the authorized application and identify potential application-level security issues.

The following areas were reviewed:

- Security headers
- Cookies
- Forms
- Input parameters
- Authentication-related behaviour
- Application responses
- Potential injection points

4. Comparison of Tools

Feature	Nikto	OWASP ZAP
Web-server assessment	Strong	Good
Security headers	Yes	Yes
Web crawling	Limited	Strong
Application testing	Limited	Strong
Information disclosure	Strong	Good
Passive analysis	Limited	Strong
Interactive testing	Limited	Strong

Conclusion of Comparison

Nikto is particularly useful for identifying web-server-level problems and common dangerous files, while OWASP ZAP provides broader web-application analysis through crawling, passive scanning, and request/response inspection.

Using both tools gives better coverage than relying on only one automated scanner.

5. Manual Validation of Finding 1 – Security Header

Suppose the scanner reports a missing security header.

Command:

```
curl -I <TARGET_URL>
```

Check whether the reported header is actually absent.

Example:

Content-Security-Policy: [not present]

Result

If the header is genuinely absent, the automated finding is confirmed.

Classification

Severity: Low/Medium

Recommendation

Configure the appropriate security header at the web-server or application level.

6. Manual Validation of Finding 2 – Information Disclosure

Suppose Nikto reports server-version disclosure.

Command:

```
curl -I <TARGET_URL>
```

If the response contains:

Server: Apache/2.4.x

then the information-disclosure finding can be considered confirmed.

Impact

The information can assist technology fingerprinting and vulnerability research.

Severity

Low.

Recommendation

Minimize unnecessary version information and keep the software patched.

7. Manual Validation of Finding 3 – Exposed Resource

If Gobuster or Nikto identifies a potentially sensitive resource, verify only the authorized laboratory resource.

Example:

```
curl -I <TARGET_URL>/backup
```

or:

```
curl -I <TARGET_URL>/admin
```

Important

Do not download, modify, delete, or exploit sensitive files. The objective is only to verify whether the resource is exposed.

Possible Results

HTTP/1.1 200 OK

may indicate that the resource is accessible.

HTTP/1.1 403 Forbidden

indicates that access is denied.

HTTP/1.1 404 Not Found

indicates that the resource is unavailable.

The actual response must be recorded from the laboratory system.

8. False Positive Analysis

Automated scanners do not always produce confirmed vulnerabilities.

A finding should be classified as:

Confirmed

The reported issue was reproduced and supported by evidence.

False Positive

The scanner reported a problem, but manual testing demonstrated that the issue was not actually present.

Requires Further Investigation

The scanner identified a potential weakness, but available evidence was insufficient to confirm or reject it safely.

9. Final Vulnerability Summary

No	Finding	Tool	Status	Severity	Impact
1	Missing security header	Nikto/ZAP	Confirmed/ To Verify	Low/Medium	Reduced browser protection
2	Server information disclosure	Nikto	Confirmed/ To Verify	Low	Technology fingerprinting
3	Exposed directory/resource	Nikto/Gobuster	Confirmed/ To Verify	Medium/High	Unauthorized information exposure
4	Input validation weakness	ZAP	Confirmed/ To Verify	Medium/High	Possible injection
5	Access-control weakness	ZAP	Confirmed/ To Verify	High	Unauthorized data access

Only findings actually observed during the laboratory exercise should be marked "Confirmed."

OVERALL CONCLUSION

The security testing simulation demonstrated the major stages of a controlled web-security assessment. Initially, reconnaissance techniques such as WHOIS, search-engine reconnaissance, Ping, Traceroute, IP configuration analysis, and Netstat/SS were used to understand the authorized environment. Nmap was then used to identify available ports, services, versions, and operating-system information.

Web-server technologies were identified using WhatWeb, while Curl was used to examine HTTP response headers and identify possible information disclosure and missing security controls. Nikto and Gobuster/Dirb were used to identify common web-server weaknesses and potentially exposed resources.

For web-application assessment, Burp Suite, OWASP ZAP, browser Developer Tools, Nikto, Nmap, and Curl were used to inspect requests, parameters,

cookies, sessions, and application behaviour. Potential SQL Injection, XSS, authentication, session-management, directory-traversal, and configuration weaknesses were considered and validated only within the authorized laboratory environment.

The HTTPS assessment focused on security headers, HTTP-to-HTTPS redirection, TLS protocols, cipher suites, and certificate configuration. The access-control assessment demonstrated the importance of server-side authorization checks and showed the difference between authentication and authorization.

Finally, automated results from Nikto and OWASP ZAP were compared with manual validation. This demonstrated that automated scanners are useful for identifying potential vulnerabilities but that every important finding should be manually reviewed before being classified as a confirmed vulnerability.

Overall, the assessment highlights the importance of secure configuration, timely patching, strong authentication and authorization, secure session management, input validation, HTTPS/TLS hardening, security headers, and continuous vulnerability assessment.

SCREENSHOTS TO INCLUDE IN THE SOFT COPY

Take screenshots of the actual commands and results from your Kali Linux laboratory environment.

1. WHOIS command and output
2. Ping result
3. Traceroute/tracepath result
4. ip addr output
5. ss -tulnp or netstat output
6. Nmap basic scan
7. Nmap -sV result
8. Nmap -sC result
9. WhatWeb result
10. Curl HTTP-header result
11. Nikto result

- 12.Gobuster/Dirb result
- 13.Burp Suite captured request
- 14.Browser Developer Tools Network tab
- 15.OWASP ZAP scan/result
- 16.SQL Injection test evidence
- 17.XSS test evidence
- 18.Session/cookie inspection
- 19.HTTPS Curl result
- 20.Nmap SSL/TLS result
- 21.ZAP access-control request comparison
- 22.Nikto and ZAP comparison
- 23.Manual validation evidence

IMPORTANT COMMAND SHEET

For quick implementation in Kali Linux, the main commands are:

whois <AUTHORIZED_DOMAIN>

ping -c 4 <TARGET_IP>

tracert <TARGET_IP>

ip addr

ss -tulnp

nmap <TARGET_IP>

`nmap -sV <TARGET_IP>`

`sudo nmap -O <TARGET_IP>`

`nmap -sC <TARGET_IP>`

`sudo nmap -sC -sV -O <TARGET_IP>`

`whatweb <TARGET_URL>`

`curl -I <TARGET_URL>`

`curl -i <TARGET_URL>`

`nikto -h <TARGET_URL>`

`nikto -h <TARGET_URL> -output nikto_report.txt`

`gobuster dir -u <TARGET_URL> -w /usr/share/wordlists/dirb/common.txt`

`dirb <TARGET_URL>`

`burpsuite`

`nmap --script ssl-enum-ciphers -p 443 <TARGET_IP>`

`nmap --script ssl-cert -p 443 <TARGET_IP>`

```
nmap --script ssl-enum-ciphers,ssl-cert -p 443 <TARGET_IP>
```

```
curl -I http://<TARGET_IP>
```

```
curl -I https://<TARGET_IP>
```

Final Note for Submission

Do **not** copy sample IP addresses, ports, versions, vulnerability results, or HTTP responses as if they were actually discovered. Replace them with the **real output from your faculty-approved lab target** and attach the corresponding screenshots. This makes the assessment technically correct and evidence-based.